Rockchip Linux updateEngine 升级方案介绍

文件标识：RK-KF-YF-348

发布版本：V2.0.0

日期：2026-06-23

文件密级：□绝密 □秘密 □内部资料 ■公开

免责声明

本文档按"现状"提供，瑞芯微电子股份有限公司（"本公司"，下同）不对本文档的任何陈述、信息和内容的准确性、可靠性、完整性、适销性、特定目的性和非侵权性提供任何明示或暗示的声明或保证。本文档仅作为使用指导的参考。

由于产品版本升级或其他原因，本文档将可能在未经任何通知的情况下，不定期进行更新或修改。

商标声明

"Rockchip"、"瑞芯微"、"瑞芯"均为本公司的注册商标，归本公司所有。

本文档可能提及的其他所有注册商标或商标，由其各自拥有者所有。

版权所有© 2026 瑞芯微电子股份有限公司

超越合理使用范畴，非经本公司书面许可，任何单位和个人不得擅自摘抄、复制本文档内容的部分或全部，并不得以任何形式传播。

瑞芯微电子股份有限公司

Rockchip Electronics Co., Ltd.

地址： 福建省福州市铜盘路软件园A区18号

网址： www.rock-chips.com

客户服务电话： +86-4007-700-590

客户服务传真： +86-591-83951833

客户服务邮箱： fae@rock-chips.com

前言

概述

本文档旨在指导工程师如何快速使用Rockchip Linux 平台升级方案，并进行二次开发。

读者对象

本文档（本指南）主要适用于以下工程师：

技术支持工程师

软件开发工程师

修订记录

日期	版本	作者	修改说明
2019-06-05	V1.0.1	HKH MLC	初始版本
2020-03-31	V1.0.2	ZLX	格式修正
2020-12-14	V1.0.3	ZLX	更新公司名称及文档格式
2021-02-18	V1.1.0	CWW	1. 更新支持 SD 卡启动来升级固件 2. 更新支持 A/B 系统升级
2023-03-16	V1.2.0	ZSQ	增加差分 OTA 支持
2026-02-26	V1.3.0	ZJH	添加 MTD/UBI 差分升级支持
2026-06-23	V2.0.0	ZJH	更新当前版本 updateEngine 参数、Buildroot 配置、OTA 升级范围、GPT/IDBlock 和写入前检查说明、支持双存储升级

目录

Rockchip Linux updateEngine 升级方案介绍

1. 简介以及代码
    1.1 简介
    1.2 代码
2. Recovery 模式
    2.1 概述
    2.2 配置和编译
    2.3 OTA升级
    2.4 日志的查看
3. 差分升级
    3.1 依赖及编译
    3.2 分区大小要求
    3.3 制作差分补丁包
    3.4 MTD/UBI 差分升级说明
4. Linux A/B 模式
    4.1 概述
    4.2 引导流程以及数据格式
        4.2.1 数据格式及存储
        4.2.2 引导流程
        4.2.3 引导流程图
    4.3 编译配置
        4.3.1 uboot
        4.3.2 Buildroot
        4.3.3 分区表
        4.3.4 固件输出
    4.4 OTA升级
    4.5 分区引导设置
        4.5.1 可引导设置
        4.5.2 A/B 分区切换
5. 升级包制作
    5.1 常用升级包类型
    5.2 Recovery 模式 OTA 包
    5.3 A/B OTA 包
    5.4 完整固件和 SD/USB 启动盘升级包
    5.5 双存储固件打包
    5.6 自定义分区打包
    5.7 打包注意事项
6. SD/USB 启动盘升级
    6.1 制作 SD 卡启动盘
    6.2 Recovery 系统模式的 SD 卡启动盘制作说明
    6.3 AB 系统模式的 SD 卡启动盘制作说明
        6.3.1 前提条件
        6.3.2 制作步骤
7. 恢复出厂设置
8. 升级程序详细说明
    8.1 参数说明
        8.1.1 --misc
        8.1.2 --update
        8.1.3 --upgrade
        8.1.4 --reboot

8.1.5 --version_url / --check
8.1.6 --image_url
8.1.7 --savepath
8.1.8 parameter 和 bootloader 升级
8.1.9 写入前检查
8.2 自定义分区升级
9. 附录
9.1 固件打包工具
9.2 updateEngine 存储写入实现说明
9.3 Misc 分区说明

1. 简介以及代码

1.1 简介

Rockchip Linux 平台支持两种启动方案，Recovery 模式和 Linux A/B 模式：

1. Recovery 模式，设备上有一个单独的分区（recovery）用于升级操作。
2. Linux A/B 模式，设备上有两套固件，可切换使用。

这两种启动模式各有优缺点，用户根据需求选择使用。

1.2 代码

Rockchip Linux 平台升级方案代码。

升级方案	代码路径	是否支持Recovery 启动模式升级	是否支持A/B 启动模式升级
updateEngine	external/recovery	支持	支持

当前版本的 `external/recovery` 会编译出两个程序：

程序	安装位置	说明
updateEngine	`/usr/bin/updateEngine`	执行升级、写 misc、切换 A/B 分区、恢复出厂设置等操作。Normal 系统和 Recovery 系统都会使用。
recovery	`/usr/bin/recovery`	Recovery 系统入口程序，只应安装到 recovery rootfs，不应安装到 normal rootfs。

2. Recovery 模式

2.1 概述

recovery 模式是在设备上多一个 recovery 分区，该分区由 kernel + resource + ramdisk 组成，主要用于升级操作。u-boot 会根据 misc 分区（详见 misc 分区章节）存放的字段来判断将要引导的系统是 normal 系统还是 recovery 系统。由于系统的独立性，所以 recovery 模式能保证升级的完整性，即升级过程被中断，如异常掉电，升级仍然能继续执行。

优点：

1. 能保证升级的完整性

缺点：

1. 系统多了一个分区，该分区仅用于升级
2. 升级过程必须重启进入 recovery 模式，不能在 normal 系统直接进行升级

分区简介：

分区名	镜像名	简介
loader	MiniLoaderAll.bin	一级loader
u-boot	uboot.img	二级loader
trust	trust.img	安全环境，如OP-TEE、ATF （有些平台上会把trust和uboot合并）
misc	misc.img	引导参数分区
recovery	recovery.img	kernel+dtb+ramdisk 组成的根文件系统
boot	boot.img	kernel+dtb
rootfs	rootfs.img	根文件系统，只读
oem	oem.img	厂商预制，可读写
userdata	userdata.img	用于数据，可读写

2.2 配置和编译

Buildroot：recovery 配置文件选择如下（make menuconfig）

```
BR2_PACKAGE_RECOVERY=y                    # 开启升级相关功能
BR2_PACKAGE_RECOVERY_USE_UPDATEENGINE=y   # 使用 updateEngine 升级程序
BR2_PACKAGE_RECOVERY_RECOVERYBIN=y        # 安装 recovery 程序到 recovery rootfs
BR2_PACKAGE_RECOVERY_UPDATEENGINEBIN=y    # 安装 updateEngine
BR2_PACKAGE_RECOVERY_NO_UI=y              # 关闭 UI；带屏产品可不配置
BR2_PACKAGE_RECOVERY_SOURCE_URL=y         # 网络下载升级
```

Buildroot：normal rootfs 配置文件选择如下（make menuconfig）

```
BR2_PACKAGE_RECOVERY=y                    # 开启升级相关功能
BR2_PACKAGE_RECOVERY_USE_UPDATEENGINE=y   # 使用 updateEngine 升级程序
BR2_PACKAGE_RECOVERY_UPDATEENGINEBIN=y    # 安装 updateEngine
BR2_PACKAGE_RECOVERY_NO_UI=y              # 关闭 UI
BR2_PACKAGE_RECOVERY_SOURCE_URL=y         # 网络下载升级
# BR2_PACKAGE_RECOVERY_RECOVERYBIN 不配置，normal rootfs 不安装 recovery 程序
```

`BR2_PACKAGE_RECOVERY_NO_UI` 只影响 recovery 二进制的 UI 编译和 libdrm/libpng 等 UI 依赖。normal rootfs 只安装 updateEngine 时配置该项，主要用于减少不必要的 UI 构建依赖；是否带 UI 应以 recovery rootfs 的配置为准。

带屏与不带屏

```
BR2_PACKAGE_RECOVERY_NO_UI=y # 关闭 UI
```

如果需要带 UI 的 recovery，不配置 `BR2_PACKAGE_RECOVERY_NO_UI`，并确保 rootfs 中包含 DRM、PNG、zlib 相关依赖。

SDK默认会开启以上配置，用户无需再次配置。源码目录位于external/recovery/，若有进行相关修改，则按照如下进行编译：

```
1. cd buildroot
2. rm -rf output/rockchip_rkxxxx/build/recovery-develop/*
3. source envsetup.sh
4. 选择某一个平台的 rootfs 配置
5. make recovery-rebuild
6. rm -rf output/rockchip_rkxxxx_recovery/build/recovery-develop/*
7. source envsetup.sh
8. 选择某一平台的 recovery 配置
9. make recovery-rebuild
10. cd SDK/
11. ./build.sh
12. 重新烧写固件
```

2.3 OTA升级

升级支持网络下载和本地升级，且必须通过 `--upgrade` 指定要升级的分区。

网络升级示例：

```
updateEngine --image_url=http://172.16.21.110:8080/recovery/update.img \
    --misc=update \
    --savepath=/userdata/update.img \
    --upgrade=boot,recovery,rootfs \
    --reboot &
```

本地升级示例：

```
updateEngine --image_url=/userdata/update.img \
    --misc=update \
    --savepath=/userdata/update.img \
    --upgrade=boot,recovery,rootfs \
    --reboot &
```

如果需要升级包中除 userdata 外的普通分区，可使用：

```
updateEngine --image_url=/userdata/update.img \
    --misc=update \
    --savepath=/userdata/update.img \
    --upgrade=all,-userdata \
    --reboot &
```

`--upgrade` 常用写法：

写法	说明
`--upgrade=boot,recovery,rootfs`	只升级 boot、recovery、rootfs。
`--upgrade=all`	升级包中除 `package-file`、`parameter`、`bootloader` 外的普通分区。
`--upgrade=all,-userdata`	升级 all 选择到的分区，但排除 userdata。推荐使用 `-userdata`，避免 shell 对 `!userdata` 做历史展开。
`--upgrade=all,parameter`	在 all 基础上额外升级 parameter/GPT。
`--upgrade=all,bootloader`	在 all 基础上额外升级 bootloader/IDBlock。
`--upgrade=all,parameter,bootloader`	常用于 SD/USB 启动盘整包升级。

> **注意：** 发布版本或量产后的 OTA 不建议默认升级 `parameter`/GPT 和 `bootloader`/IDBlock。GPT 或 IDBlock 升级过程中断电、写入失败或布局配置错误，可能导致设备无法从存储介质启动，需要通过线刷或 Maskrom 方式恢复。

流程介绍：

1. 固件版本比较（可选，`--version_url` 或 `--check`）。
2. 下载固件或使用本地固件（`--image_url`），并保存到本地（`--savepath`）。
3. 在 normal 系统中，如果 `--upgrade` 包含 `recovery`，先升级 recovery 分区。
4. 将剩余升级参数写入 misc/BCB。
5. 重启（`--reboot`）进入 recovery 系统。
6. recovery 系统读取 misc/BCB，执行剩余分区升级。
7. 升级成功，清理 misc/BCB 和升级包，重启进入 normal 系统。

可缺省参数：

1. `--version_url`：远程地址或本地地址，没有设置该参数，则不会进行版本比较。
2. `--savepath`：固件保存地址。建议传入 `/userdata/update.img` 或外部介质路径。
3. `--reboot`：升级 recovery 完成后，重启进入 recovery 模式。没有传入该参数，则需要用户手动重启。

注意事项：

1. `--upgrade` 为必选参数，除 SD/USB 自动升级流程外，不传会直接报错。
2. `parameter` 和 `bootloader` 不会被 `all` 自动包含，必须显式写入 `--upgrade`。
3. 如果升级包保存在 `/userdata/update.img`，本次升级不要同时升级 `userdata`，否则写入前检查会发现

userdata 仍被挂载并中止升级。
4. `--upgrade=boot.recovery` 这类格式错误不会被自动拆分成 boot 和 recovery，应写成 `--upgrade=boot,recovery`。

2.4 日志的查看

1. 通过串口查看升级日志。
2. 通过 `/userdata/recovery/` 目录查看持久化日志。

升级之后，可以在设备上查看如下日志文件：

```
cat /userdata/recovery/log
cat /userdata/recovery/last_log
```

其中 `log` 为追加日志，`last_log` 为最近一次 recovery 运行日志。

3. 差分升级

Linux SDK 中提供了`tools/linux/Linux_Diff_Firmware/`用来做差分升级包，支持的功能有：

- uboot/trust/kernel 分区
- rootfs 分区支持 squashfs（包括 block 设备和 UBI 卷）
- 使用 mtd 驱动的 nand flash 系统（支持 raw MTD 分区和 UBI SquashFS 分区的差分升级）

尚未支持的功能有：

- ext2/ext4格式的文件系统
- 加密分区
- A/B模式

3.1 依赖及编译

PC端依赖的工具有： bsdiff，md5sum，ubireader_extract_images。安装方法如下：

```
sudo apt install bsdiff
pip install ubi_reader
```

其中 `ubi_reader` 用于从 UBI image 中提取 squashfs 数据来制作差分包。如果 rootfs.img 为 UBI image 格式，则必须安装此工具，否则差分包制作会失败。

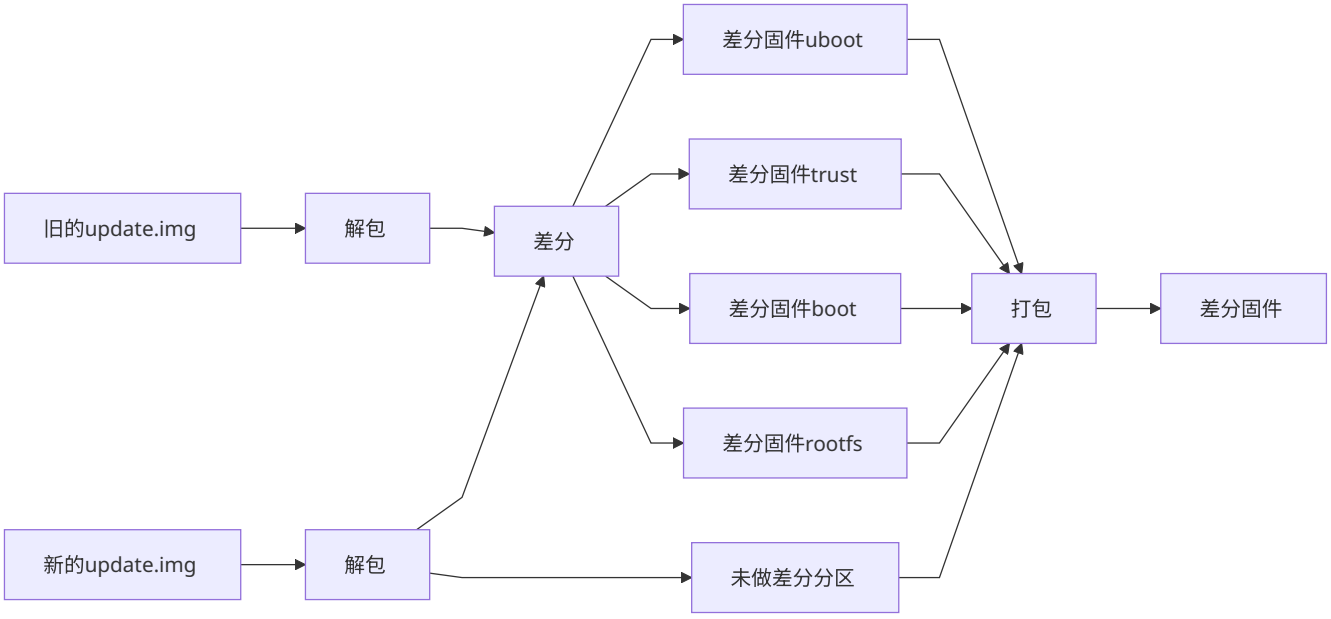
3.2 分区大小要求

在板端，对 ddr 及分区大小的要求：

- ddr 无要求，与差分补丁包大小无关
- 空闲分区大小有要求：空闲分区应能同时容纳 `update_diff.img` 和差分应用过程中所需的完整新 `rootfs.img` 数据
    - 如果 `update_diff.img` 保存在 `/userdata/` 目录下
    - `/userdata/` 的空闲空间建议不小于 `update_diff.img` 和完整新 `rootfs.img` 两者之和

3.3 制作差分补丁包

通过对新、旧 OTA 固件（update.img）解包、差分、打包，制作补丁包。



注意

- 新的 update.img 需要为 OTA image，而不是下载固件使用的 update.img，即只包含要升级的分区
- 旧的固件，建议使用包含所有分区的 update.img，至少应该包含需要差分的分区
- 旧的固件务必要与目标机器上的固件相同

使用示例：

```
cd tools/linux/Linux_Diff_Firmware/
./mk-diff-ota.sh update_old.img update_new.img update_diff.img
```

生成的 update_diff.img 即可用于OTA升级。

3.4 MTD/UBI 差分升级说明

对于使用 MTD 驱动的 NAND Flash 系统，差分升级支持以下两种分区类型：

- **Raw MTD 分区**（如 boot）：升级时从 MTD 设备读取旧数据，应用 bsdiff 补丁后通过 flash_erase + nandwrite 写回。
- **UBI SquashFS 分区**（如 rootfs）：升级时先 attach UBI 设备，从 UBI 卷中读取旧的 squashfs 数据，应用 bsdiff 补丁生成新的 squashfs，再通过 ubiupdatevol 写入 UBI 卷。

制作差分包时，如果 rootfs.img 为 UBI image 格式，脚本会自动使用 `ubireader_extract_images` 从 UBI image 中提取 squashfs 数据，再与新的 squashfs 进行差分。这样保证了差分基线数据与设备端 UBI 卷中存储的数据一致。

4. Linux A/B 模式

4.1 概述

Linux A/B，即准备两份独立的系统固件，分别存放在 flash 上，系统可以从其中一个 slot 启动，如果当前 slot 启动失败，可以从另外一个 slot 启动，在Normal模式下直接升级系统，无需进入系统升级模式，只需重启系统即可进入升级过的系统。

Linux A/B 由于有两个引导 slot，所以具有以下**优点**：

1. 升级无需重启进入升级模式，即机器可以在当前系统上直接进行升级。
2. 防止升级失败后设备无法启动；如果新版本启动失败，设备可以回退到当前可用版本。
3. 当前系统如果由于一些误操作被破坏掉，系统会自动切换到另外一个 slot 上。

缺点：

1. Linux A/B 有两个 slot，所以会增加 flash 上系统固件的占用率。

分区：

分区名	镜像名	简介
loader	MiniLoaderAll.bin	一级 loader，机器备份 4-5 份
uboot_a	uboot.img	二级 loader，机器备份2份，可修改 u-boot/make.sh 来修改备份份数
uboot_b	uboot.img	uboot_a 的备份分区
trust	trust.img	安全相关，机器备份2份，可修改u-boot/make.sh来修改备份份数 （目前平台上会把 trust 和 uboot 合并打包在一起）
misc	misc.img	引导参数分区
boot_a	boot.img	kernel+dtb，引导 system_a
boot_b	boot.img	kernel+dtb，引导 system_b
system_a	rootfs.img	根文件系统
system_b	rootfs.img	根文件系统
oem	oem.img	厂商预制，可读写
userdata	userdata.img	用于数据，可读写，无备份

4.2 引导流程以及数据格式

4.2.1 数据格式及存储

存储位置为 misc 分区偏移 2K 位置，AvbABSlotData 和 AvbABData 数据结构如下：

AvbABSlotData：存储 slot_a 和 slot_b

数据名称	数据作用
unsigned char priority	分区优先级，0~15，0 为不可自动，15 为最高优先级
unsigned char tries_remaining	尝试启动次数，最高为 7 次，可修改
unsigned char successful_boot	0：不可启动，1：可启动
unsigned char is_update:1	0：升级失败，1：升级成功，后 7 位为保留数据

AvbABData：slot_a 和 slot_b 的引导信息

数据名称	数据作用
unsigned char magic[AVB_AB_MAGIC_LEN]	结构体头部信息：\0AB0
unsigned char version_major	版本信息
unsigned char version_minor	版本信息
unsigned char reserved1[2]	保留数据
AvbABSlotData slots[2]	分区引导信息
unsigned char last_boot	上一次成功启动的分区：0->slot_a，1->slot_b
unsigned char reserved2[11]	保留数据
unsigned char crc32	Crc 数据校验

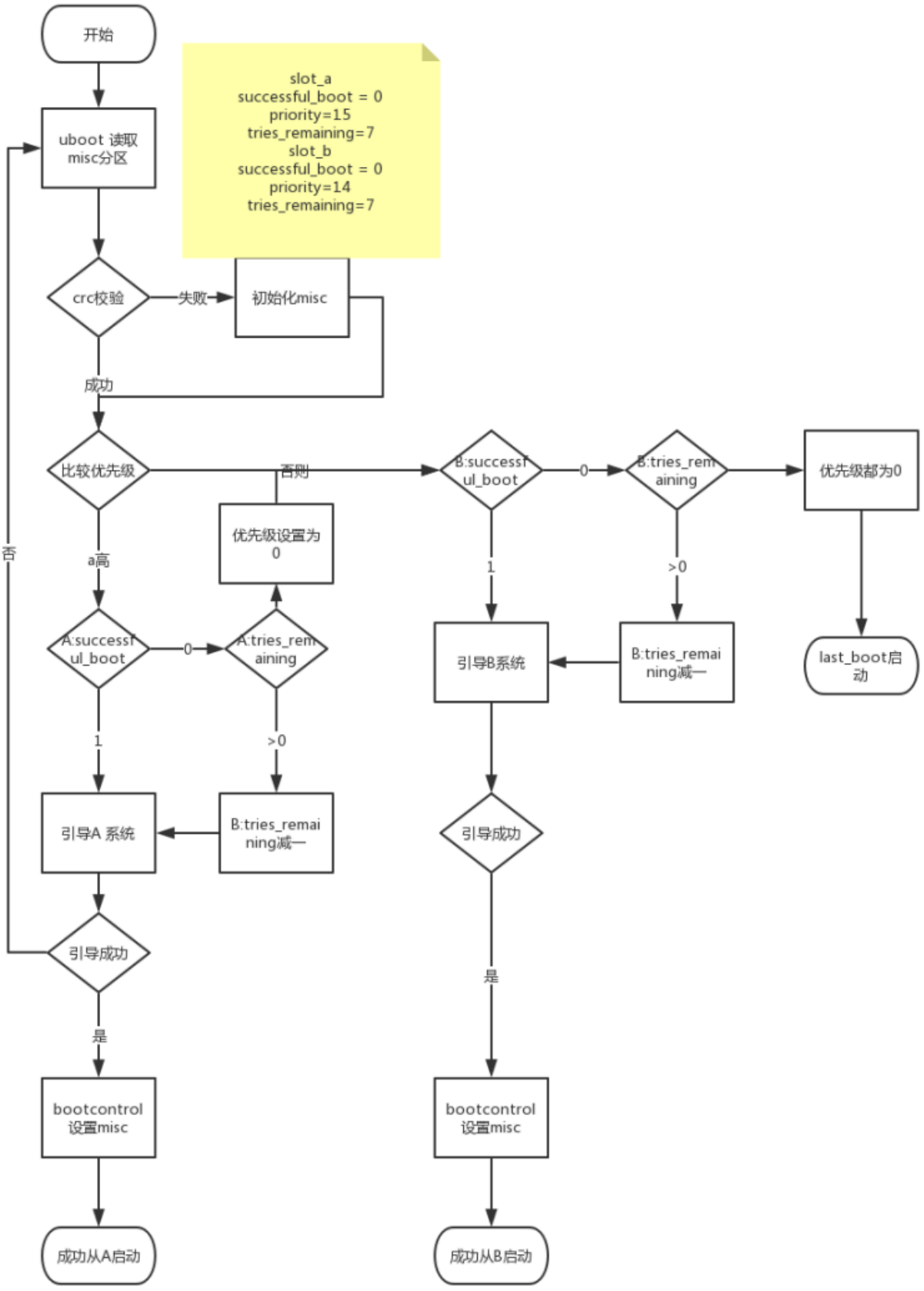
4.2.2 引导流程

根据上层 bootcontrol 程序的设置方式，可分为两种引导方式 successful_boot 和 reset retry。两种模式的对比如下：

模式	优点	缺点	成功启动设置的数据（A启动）	升级时设置的数据（A启动，B升级）
Successful boot	只要正常启动系统，不会回退到旧版本固件	设备长时间工作后，如果存储某些颗粒异常，会导致系统一直重启	tries_remaining=0 successful_boot=1 last_boot=0	A:priority=14 B:priority=15
Reset retry	始终保持 retry 机制，可以应对存储异常问题	1.机器会回到旧的版本上，可能出现版本不可控问题 2.如果因为用户误操作，retry尝试次数过了，会误判为当前分区为可启动	tries_remaining=7 last_boot=0	A:priority=14 B:priority=15

> **建议：** NAND Flash 产品优先使用 successful boot 模式。retry 模式在每次启动或掉电重试过程中会更新 misc 分区，长期频繁擦写可能增加 NAND 坏块风险。

4.2.3 引导流程图


开始
slot_a
successful_boot = 0
priority=15
tries_remaining=7
slot_b
successful_boot = 0
priority=14
tries_remaining=7
uboot 读取misc分区
crc校验
失败
初始化misc
成功
比较优先级
否则
B:successful_boot
0
B:tries_remaining
优先级都为0
last_boot启动
优先级设置为0
a高
1
>0
A:successful_boot
0
A:tries_remaining
引导B系统
B:tries_remaining减一
1
>0
引导A 系统
B:tries_remaining减一
引导成功
否
是
bootcontrol设置misc
成功从A启动
引导成功
是
bootcontrol设置misc
成功从B启动

4.3 编译配置

4.3.1 uboot

defconfig 增加如下配置，如 u-boot/configs/rk-ab.config

```
CONFIG_SPL_AB=y
CONFIG_AVB_LIBAVB=y
CONFIG_AVB_LIBAVB_AB=y
CONFIG_AVB_LIBAVB_ATX=y
CONFIG_AVB_LIBAVB_USER=y
CONFIG_RK_AVB_LIBAVB_USER=y
CONFIG_ANDROID_AB=y
```

4.3.2 Buildroot

```
BR2_PACKAGE_RECOVERY=y                    # 开启升级功能
BR2_PACKAGE_RECOVERY_BOOTCONTROL=y        # 开启引导控制脚本
BR2_PACKAGE_RECOVERY_SUCCESSFUL_BOOT=y    # 默认使用 successful boot 模式
BR2_PACKAGE_RECOVERY_USE_UPDATEENGINE=y   # 使用 updateEngine 升级程序
BR2_PACKAGE_RECOVERY_UPDATEENGINEBIN=y    # 兼容旧配置
BR2_PACKAGE_RECOVERY_NO_UI=y              # 关闭 UI
BR2_PACKAGE_RECOVERY_SOURCE_URL=y         # http 网络下载升级
```

A/B normal rootfs 不需要安装 `recovery` 程序，因此不要配置 `BR2_PACKAGE_RECOVERY_RECOVERYBIN`。如果添加了配置，自启 recovery 程序，会导致系统不断重启

默认建议使用 successful boot 模式。若产品需要 reset retry 机制，可将 `BR2_PACKAGE_RECOVERY_SUCCESSFUL_BOOT` 替换为 `BR2_PACKAGE_RECOVERY_RETRY=y`，但 NAND Flash 产品应评估频繁更新 misc 分区对寿命和坏块风险的影响。

注意：设置完成之后，须进行重新编译，如下：

```
1. cd buildroot
2. source envsetup.sh
3. 选择某一个平台的 rootfs 配置
4. make recovery-rebuild
```

4.3.3 分区表

创建编辑 parameter 分区信息示例如下：

```
CMDLINE:
mtdparts=:0x00004000@0x00004000(uboot_a),0x00004000@0x00008000(uboot_b),0x00002000@0x0000c00
0(misc),0x00020000@0x0000e000(boot_a),0x00020000@0x0002e000(boot_b),0x00010000@0x0004e000(ba
ckup),0x00e00000@0x0005e000(system_a),0x00e00000@0x00e5e000(system_b),0x00040000@0x01c5e000(
oem),-@0x01c9e000(userdata:grow)
```

A/B 系统下 rootfs 分区必须命名为 `system_a/system_b`。当前 U-Boot A/B 逻辑识别的是 `system` 分区名，因此设备端升级命令使用 `--upgrade=boot,system`。

SDK/device/rockchip/rkxx 下相应的 defconfig，设置 parameter 分区表，如下：

```
RK_AB_UPDATE=y
RK_PARAMETER="parameter-ab.txt"
```

4.3.4 固件输出

选择相应的板级配置后，执行编译命令：

```
./build.sh
```

即可生成如下固件：

```
rockdev/
├── boot.img
├── MiniLoaderAll.bin
├── misc.img
├── oem.img
├── parameter.txt
├── rootfs.img
├── uboot.img
├── update-ab.img
├── update.img
└── userdata.img
```

> **说明：** 若 defconfig 中配置了 `RK_SDCARD_UPDATE`，且 `recovery.img` 已编译生成，SDK 将在打包阶段自动生成 `update-sd.img`，用于制作 SD 卡升级启动盘。详见 AB 系统模式的 SD 卡启动盘制作说明。

升级固件

```
./build.sh ota-updateimg
```

rockdev 和 IMAGE 目录下会生成 update_ota.img，用于 OTA 升级。该 IMAGE 包包含 boot.img 和 rootfs.img。可根据实际需求执行 `./build.sh edit-ota-package-file` 修改需要打包的固件文件。如下图：

```
# NAME          Relative path
#
#HWDEF         HWDEF
package-file   package-file
bootloader     Image/MiniLoaderAll.bin
parameter      Image/parameter.txt
uboot_a        Image/uboot.img
boot_a         Image/boot.img
system_a       Image/rootfs.img
oem            Image/oem.img
```

烧写固件

rockdev 和 IMAGE 目录下会生成 update-ab.img，该固件用于烧写。可根据实际需求执行 `./build.sh edit-package-file` 修改需要打包的固件文件。如下图：

```
# NAME           Relative path
#
#HWDEF          HWDEF
package-file    package-file
bootloader      Image/MiniLoaderAll.bin
parameter       Image/parameter.txt
misc            Image/misc.img
uboot_a         Image/uboot.img
uboot_b         Image/uboot.img
boot_a          Image/boot.img
boot_b          Image/boot.img
system_a        Image/rootfs.img
system_b        Image/rootfs.img
oem             Image/oem.img
```

4.4 OTA升级

Linux A/B 模式在 Normal 系统中直接执行升级，不需要重启进入 recovery 系统。升级时需要通过 `--upgrade` 指定分区名称。

网络升级：

```
updateEngine --image_url=http://172.16.21.110:8080/linuxab/update_ota.img \
    --update \
    --upgrade=boot,system \
    --reboot
```

本地升级：

```
updateEngine --image_url=/userdata/update_ota.img \
    --update \
    --upgrade=boot,system \
    --reboot
```

流程介绍：

1. 固件版本比较（可选）。
2. 下载固件（`--image_url`），并保存到本地（`--savepath`，可选）。
3. 根据当前 slot 选择另一个 slot 中的目标分区进行升级。例如当前从 A 启动，则升级 `boot_b`、`system_b`。
4. 设置升级后的 slot 为下次引导分区。
5. 重启。
6. 尝试引导升级后的分区。
7. 新系统启动成功后，由 S99_bootcontrol 脚本执行 `updateEngine --misc=now` 标记启动成功。

可缺省参数：

1. `--version_url` 或 `--check`：没有设置该参数，则不会进行版本比较。
2. `--savepath`：固件保存地址，缺省时直接使用 `--image_url` 指定的本地文件，网络下载时建议指定保存路径。
3. `--reboot`：升级完成后重启；未传入则需要用户手动重启。

注意事项：

1. A/B 系统下 rootfs 分区必须命名为 `system_a/system_b`，升级命令使用 `--upgrade=boot,system`。
2. `boot`、`system` 这类基础名称可以匹配 `boot_a/boot_b`、`system_a/system_b`。
3. A/B 模式只会升级非当前 slot，当前正在启动的 slot 会被跳过。
4. `all` 不包含 `parameter` 和 `bootloader`。A/B OTA 通常不建议升级 `parameter`。

4.5 分区引导设置

4.5.1 可引导设置

通过misc设置当前分区为可引导分区，要在 system 成功引导之后执行，标记系统成功启动，参考如下脚本

```
$external/recovery/scripts$ cat S99_bootcontrol
case "$1" in
  start)
        /usr/bin/updateEngine --misc=now
        ;;
  stop)
        printf "stop finished\n"
        ;;
  *)
        echo "Usage: $0 {start|stop}"
        exit 1
        ;;
esac
exit 0
```

4.5.2 A/B 分区切换

```
updateEngine --misc=other --reboot
```

流程介绍：

1. 往 misc 偏移 2K 位置更新 A/B 引导信息，设置下一次启动到另一个 slot
2. 重启

可缺省参数：

1. --reboot，缺省则机器不会立即重启，在下一次重启才会生效

注意：updateEngine 程序在 OTA 升级结束之后会自动设置，无需重复设置。

5. 升级包制作

升级包由 `package-file` 描述需要打包的镜像。`package-file` 第一列是升级 item 名称，第二列是镜像文件相对路径。设备端执行升级时，`--upgrade` 指定的是第一列中的 item 名称。在 SDK 下可以使用 `./build.sh edit-package-file` 和 `./build.sh edit-ota-package-file` 编辑 package-file 文件。

5.1 常用升级包类型

升级包	用途	常用生成方式	典型内容
`update.img`	量产烧写或完整固件	`./build.sh updateimg`	`parameter`、`bootloader`、`misc`、`boot`、`recovery`、`rootfs`、`userdata` 等。
`update_ota.img`	normal 系统或 recovery 系统 OTA	`./build.sh ota-updateimg`	只包含本次需要升级的分区，例如 `boot`、`recovery`、`rootfs`。
`update-ab.img`	A/B 完整固件烧写	`./build.sh updateimg`	`boot_a`、`boot_b`、`system_a`、`system_b` 等 A/B 分区。
`update-sd.img`	A/B 产品制作 SD 启动盘	`RK_SDCARD_UPDATE=y` 后由 `./build.sh updateimg` 生成	带 recovery 分区，只用于 SD 卡启动引导。
`update_diff.img`	差分 OTA	`tools/linux/Linux_Diff_Firmware/mk-diff-ota.sh`	差分补丁和未差分升级的镜像。

5.2 Recovery 模式 OTA 包

Recovery 模式 OTA 包建议只打包本次需要升级的分区。示例：

```
# NAME          Relative path
package-file    package-file
boot            Image/boot.img
recovery        Image/recovery.img
rootfs          Image/rootfs.img
```

设备端升级命令：

```
updateEngine --image_url=/userdata/update_ota.img \
    --misc=update \
    --savepath=/userdata/update_ota.img \
    --upgrade=boot,recovery,rootfs \
    --reboot &
```

如果升级包保存在 `/userdata`，不要在同一次升级中选择 `userdata`。否则写入前检查会发现目标分区仍处于挂载状态并中止升级。

5.3 A/B OTA 包

A/B OTA 包通常只包含非共享系统分区，例如 `boot_a`、`system_a`。updateEngine 会根据当前 slot 自动选择非当前 slot。

```
# NAME          Relative path
package-file    package-file
boot_a          Image/boot.img
system_a        Image/rootfs.img
```

设备端升级命令：

```
updateEngine --image_url=/userdata/update_ota.img \
    --update \
    --upgrade=boot,system \
    --reboot
```

A/B 系统下 rootfs 分区必须命名为 `system_a/system_b`，因此设备端命令使用 `--upgrade=boot,system`。

5.4 完整固件和 SD/USB 启动盘升级包

完整固件可以包含 `parameter` 和 `bootloader`，主要用于量产烧写、SD/USB 启动盘升级或明确需要重写分区表/IDBlock 的维护场景。

```
# NAME          Relative path
package-file    package-file
bootloader      Image/MiniLoaderAll.bin
parameter       Image/parameter.txt
misc            Image/misc.img
boot            Image/boot.img
recovery        Image/recovery.img
rootfs          Image/rootfs.img
userdata        Image/userdata.img
```

SD/USB 启动盘升级由 recovery 自动调用 updateEngine，默认升级范围为：

```
--upgrade=all,parameter,bootloader
```

量产后的 OTA 不建议默认升级 `parameter` 和 `bootloader`。GPT 和 IDBlock 均属于启动关键数据，如果升级过程中断电、写入失败或布局配置错误，设备可能无法从存储介质启动，需要通过线刷或 Maskrom 方式恢复。

5.5 双存储固件打包

双存储产品通常包含 boot storage 和 main storage 两类介质。boot storage 用于保存引导相关固件，例如 SPI NOR 中的 IDBlock、U-Boot 和 boot storage 自身的 `parameter.txt`；main storage 用于保存系统分区，例如 SATA/NVMe/eMMC 中的 `boot`、`rootfs`、`userdata` 和 main storage 自身的 `parameter.txt`。

常见组合包括 NOR + SATA、NOR + NVMe、NOR + eMMC 等。具体存储启动原理、U-Boot 配置和分区布局，可参考《Rockchip_Developer_Guide_Dual_Storage_CN.pdf》。

双存储完整固件建议按存储介质分别准备镜像目录，再合并为一个 `update.img`。目录示例如下：

```
.
├── Image/                      # main storage 固件
│   ├── parameter.txt           # main storage 分区表
│   ├── boot.img
│   ├── rootfs.img
│   └── userdata.img
├── Image-nor/                  # boot storage 固件
│   ├── parameter.txt           # boot storage 分区表
│   ├── MiniLoaderAll.bin
│   └── uboot.img
├── afptool
└── rkImageMaker
```

main storage 的 `package-file` 示例：

```
# NAME          Relative path
package-file    package-file
parameter       Image/parameter.txt
boot            Image/boot.img
rootfs          Image/rootfs.img
userdata        Image/userdata.img
```

boot storage 的 `package-file` 示例：

```
# NAME          Relative path
package-file    package-file
bootloader      Image-nor/MiniLoaderAll.bin
parameter       Image-nor/parameter.txt
uboot           Image-nor/uboot.img
```

打包流程建议如下：

1. 分别使用 `afptool` 将 main storage 和 boot storage 的 `package-file` 打包为两个子固件。
2. 分别使用 `rkImageMaker` 为两个子固件写入对应的 storage 类型，例如 main storage 使用 `-storage:emmc`、boot storage 使用 `-storage:spinor`。实际参数以平台存储类型为准。
3. 使用 `rkImageMaker -merge` 将两个子固件合并为最终 `update.img`。

示例流程：

```
./afptool -pack ./ Image/update.img package-file-main
./rkImageMaker -RK3588 Image/MiniLoaderAll.bin Image/update.img main_update.img -
os_type:androidos -storage:emmc

./afptool -pack ./ Image-nor/update.img package-file-nor
./rkImageMaker -RK3588 Image-nor/MiniLoaderAll.bin Image-nor/update.img nor_update.img -
os_type:androidos -storage:spinor

./rkImageMaker -merge ./update.img ./main_update.img ./nor_update.img
```

> **说明：** 上述命令用于说明双存储打包流程，实际平台的 vendor id、storage 类型、loader 文件名和脚本路径以 SDK 打包脚本为准。需要完整脚本示例时，可参考《Rockchip_Developer_Guide_Dual_Storage_CN.pdf》中的"双存储方案固件打包"章节。

使用 updateEngine 升级双存储固件时，`--upgrade` 仍然使用升级包中的 item 名称。示例：

```
updateEngine --image_url=/udisk/update.img \
    --misc=update \
    --savepath=/userdata/update.img \
    --upgrade=bootloader,uboot,boot,rootfs \
    --reboot &
```

双存储 OTA 不建议默认升级 `parameter` 和 `bootloader`。如果确实需要升级，应确认 boot storage 和 main storage 的分区表同时匹配，并确认未升级分区的起始地址不发生变化。双存储固件中可能同时包含 boot storage 和 main storage 的 `parameter`。这类固件更适合用于 SD/USB 启动盘完整升级或量产维护场景。

5.6 自定义分区打包

自定义分区不需要修改 updateEngine 代码。只要设备分区表和升级包 item 名称一致，即可通过 `--upgrade=<name>` 升级。

```
# NAME          Relative path
package-file    package-file
factory         Image/factory.img
```

设备端执行：

```
updateEngine --image_url=/userdata/update_factory.img \
    --update \
    --upgrade=factory \
    --reboot
```

5.7 打包注意事项

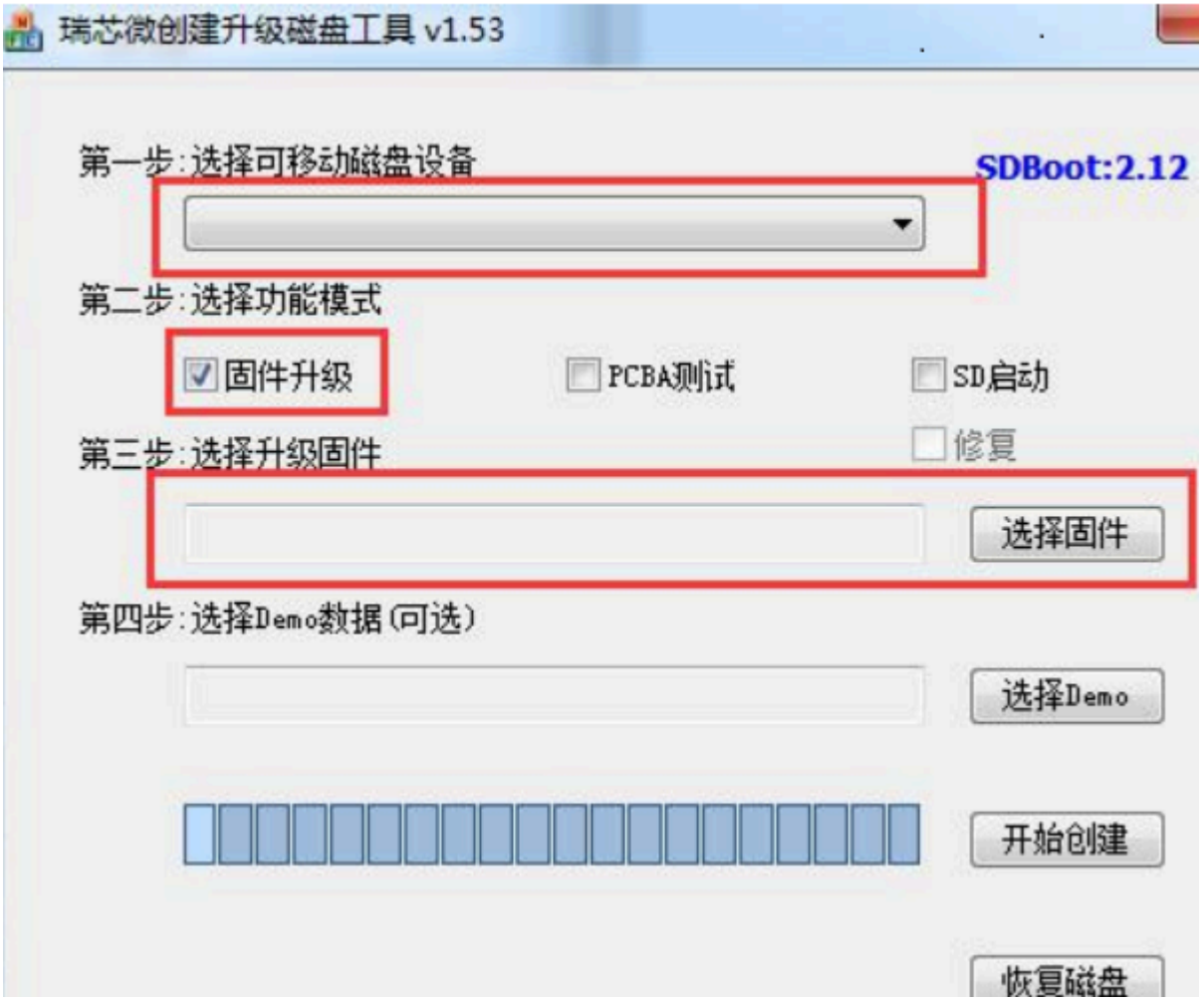
1. `package-file` 中的名称必须与设备端分区名一致。block 设备依赖 `/dev/block/by-name/<name>`，MTD 设备依赖 `/proc/mtd` 中的分区名。
2. `--upgrade=all` 不包含 `parameter` 和 `bootloader`，需要升级时必须显式写成 `--upgrade=all,parameter` 或 `--upgrade=all,bootloader`。
3. 修改分区表时，需要确认未升级的分区起始地址不发生变化；如果某个分区起始地址变化，应同时把该分区镜像加入升级包并写入 `--upgrade`。
4. 差分包要求目标设备当前固件与旧包基线一致，否则补丁无法可靠应用。
5. 网络升级时建议指定 `--savepath`，避免下载到空间不足或重启后丢失的位置。

6. SD/USB 启动盘升级

SD 卡启动盘升级指将通过 SDDiskTool 制卡工具制作的 SD 卡插入到机器中进行升级。USB 启动盘升级使用相同的 updateEngine 流程，只是启动介质和 `--update` 参数不同。

6.1 制作 SD 卡启动盘

如图所示，使用工程目录 tools\windows\SDDiskTool 中的 SD 卡启动盘升级制作工具制作 SD 卡启动盘。



在“选择固件”中选择打包好的 update.img 文件。

所有准备工作完成后，点击开始创建按钮，如果创建成功，会弹窗提示。

此时 SD 卡中根目录会存在两个文件，其中选择升级的固件 update.img，会被命名为 sdupdate.img。

所有准备工作做好后，设备中插入 SD 卡，并重新上电。

文件系统格式说明：

SDDiskTool 制卡时默认将 SD 卡格式化为 FAT32 文件系统。FAT32 格式存在以下限制：

- 单个文件大小不能超过 4GB
- SD 卡容量最大支持 32GB

当升级包大小超过 4GB 或 SD 卡容量超过 32GB 时，使用 FAT32 格式制卡将会失败。此时需要修改 SDDiskTool 目录下的 `config.ini` 配置文件，将文件系统格式更改为 exFAT 或其他支持大文件的格式：

```
USER_DISK_FS=EXFAT
```

> **注意：** 更改文件系统格式后，需要确保设备端 kernel 已开启对应文件系统的支持（如 `CONFIG_EXFAT_FS=y`），否则设备将无法挂载 SD 卡。

SD 启动卡的格式参考《Rockchip_Developer_Guide_UBoot_Nextdev_CN.pdf》文档。

6.2 Recovery 系统模式的 SD 卡启动盘制作说明

Recovery 系统模式的 SD 卡启动盘制作只要把 update.img 用 SDDiskTool 工具直接制作成 sdupdate.img 即可。

6.3 AB 系统模式的 SD 卡启动盘制作说明

SD 卡启动盘升级依赖 Recovery 模式引导。由于 AB 系统分区表不包含 recovery 分区，无法直接通过 SD 卡引导进入 Recovery 模式执行升级。为解决该问题，SDK 在打包阶段基于非 AB 分区表（包含 recovery 分区）自动生成 `update-sd.img`，专用于 SD 卡启动盘的制作。

6.3.1 前提条件

1. defconfig 中配置了 `RK_SDCARD_UPDATE=y`
2. `recovery.img` 已编译生成（可通过 `./build.sh recovery` 构建）

配置示例：

```
RK_PARAMETER="parameter-ab.txt"
RK_AB_UPDATE=y
RK_SDCARD_UPDATE=y
```

满足上述条件后，执行 `./build.sh updateimg` 将自动生成以下固件：

- **update-ab.img**：基于 AB 分区表打包，包含完整的双分区系统镜像，用于设备烧录。
- **update-sd.img**：基于非 AB 分区表打包（包含 recovery 分区），专用于 SD 卡启动盘制作。

> **说明：** 若未配置 `RK_SDCARD_UPDATE`，将跳过 `update-sd.img` 的生成。

6.3.2 制作步骤

1. 使用 SDDiskTool 工具加载 `update-sd.img` 制作 SD 卡启动盘。
2. 将 `update-ab.img` 复制至 SD 卡根目录，并重命名为 `sdupdate.img`。
3. 将 SD 卡插入目标设备并重新上电，设备将自动进入 Recovery 模式执行升级。

制作完成后，SD 卡根目录结构如下：

```
rksdfw.tag
sd_boot_config.config
sdupdate.img            # 由 update-ab.img 重命名
```

> **说明：** `update-sd.img` 中的 recovery 分区仅用于 SD 卡启动盘引导升级流程，实际写入目标设备的固件（`sdupdate.img`）使用标准 AB 分区表，升级后目标设备上不会包含 recovery 分区。

7. 恢复出厂设置

我们把可以读写的配置文件保存在 userdata 分区，出厂固件会默认一些配置参数，用户使用一段时间后会生成或修改配置文件，有时用户需要清除这些数据，我们就需要恢复到出厂配置。

SDK 实现：

功能键 RECOVERY + VOLUMEUP 触发恢复出厂配置，代码请参考：

buildroot/board/rockchip/rk3308/fs-overlay/etc/input-event-daemon.conf

buildroot/board/rockchip/rk3308/fs-overlay/usr/sbin/factory_reset_cfg

```
updateEngine --misc=wipe_userdata --reboot
```

流程介绍：

1. 往 misc 分区偏移 6K 位置写入格式化命令
2. 重启（--reboot）
3. S21mountall.sh 识别misc中有格式化命令
4. 格式化userdata

可缺省参数：

1. --reboot 如果没有传入该参数，则在机器下次重启后才会恢复出厂设置。

8. 升级程序详细说明

8.1 参数说明

updateEngine 主要包含升级分区、写 Misc 配置、恢复出厂设置等功能，支持命令参数如下：

```
*** updateEngine v2.0.0 ***
--misc=now                    Linux A/B mode: Setting the current partition to bootable.
--misc=other                  Linux A/B mode: Setting another partition to bootable.
--misc=display                Display misc and A/B information.
--misc=update                 Recovery mode: Setting the partition to be upgraded.
--misc=wipe_userdata          Format data partition.
--misc_custom=read|write|clean Read/write/clean custom misc command line.
--upgrade=<all|name,...>      Set partitions to be upgraded.
--reboot                      Restart the machine at the end of the program.
--version_url=<url>           The path to the version file.
--check=<url>                 Same as version check input.
--image_url=<url>             Path to upgrade firmware.
--savepath=<path>             Save the update.img to path.
--version                     Display updateEngine version.
--help                        Display help information.
```

8.1.1 --misc

`now`：供 Linux A/B 模式使用，将当前分区设置为可引导分区。

注意：`S99_bootcontrol` 脚本在开机最后阶段会运行该命令，将当前分区设置为可引导分区，需要开启：

```
BR2_PACKAGE_RECOVERY_BOOTCONTROL=y
```

`other`：供 Linux A/B 模式使用，将另外一个分区设置为升级完成分区，重启之后会尝试从另外一个分区引导。

注意：如果使用 updateEngine 升级，在升级结束之后会自动设置，无需重复设置。

`update`：供 Recovery 模式使用。在 normal 系统中，先升级 recovery 分区，然后把剩余升级命令写入 misc/BCB；重启进入 recovery 系统后继续升级其余分区。

`display`：调试使用，显示 misc 分区的数据结构。

`wipe_userdata`：写入恢复出厂设置命令。

8.1.2 --update

不带参数：主要供 Linux A/B 使用，在当前系统下直接升级非当前 slot。

8.1.3 --upgrade

设置将要升级的分区。

写法	说明
`--upgrade=boot,recovery,rootfs`	升级指定分区。
`--upgrade=all`	升级包中除 `package-file`、`parameter`、`bootloader` 外的普通分区。
`--upgrade=all,-userdata`	升级 all 选择到的分区，但排除 userdata。
`--upgrade=all,parameter`	在 all 基础上额外升级 parameter/GPT。
`--upgrade=all,bootloader`	在 all 基础上额外升级 bootloader/IDBlock。
`--upgrade=all,parameter,bootloader`	常用于 SD/USB 启动盘整包升级。

说明：

1. `--upgrade` 除 SD/USB 自动升级流程外为必选参数。
2. `parameter` 和 `bootloader` 是特殊升级项，必须显式指定，`all` 不会默认包含。
3. `-name` 表示从升级范围中排除某个分区，例如 `--upgrade=all,-userdata`。
4. `!name` 也表示排除，但交互式 shell 可能把 `!` 当作历史展开，因此推荐使用 `-name`。

5. A/B 模式中，`boot` 可以匹配 `boot_a/boot_b`，`system` 可以匹配 `system_a/system_b`，updateEngine 会自动选择非当前 slot。

8.1.4 --reboot

updateEngine 运行成功之后，机器重启。

8.1.5 --version_url / --check

如果有传入路径，升级之前会与 `/etc/version` 文件中的 `RK_VERSION=` 版本值进行比较。

本地路径：从本地文件读取版本号。

远程路径：从远程下载版本文件，远程版本文件格式必须跟 `/etc/version` 一致。

8.1.6 --image_url

设置升级固件的路径，可为远程或本地路径。

8.1.7 --savepath

设置保存固件的位置。如果没有传入且升级的固件路径为远程地址，建议由调用方明确指定，例如 `/userdata/update.img`。

8.1.8 parameter 和 bootloader 升级

> **注意：** 发布版本或量产后的 OTA 不建议默认升级 `parameter`/GPT 和 `bootloader`/IDBlock。这两类数据属于设备启动关键数据，升级过程中断电、写入失败或布局配置错误，可能导致设备无法从存储介质启动，需要通过线刷或 Maskrom 方式恢复。只有在明确需要调整分区表、更新 IDBlock 或制作 SD/USB 完整升级介质时，才建议显式启用。

`parameter` 对应 GPT 升级，只在编译 `RK_GPT_UPDATE` 时可用。升级时必须显式指定：

```
updateEngine --image_url=/userdata/update.img --update --upgrade=all,parameter
```

GPT 写入时会根据 `parameter.txt` 生成标准 GPT 数据，并写入两份表：

1. primary GPT：写在目标整盘设备起始位置。
2. backup GPT：写在目标整盘设备尾部，结束位置与设备可用容量末尾对齐。

backup GPT 的位置依赖目标介质的逻辑块大小和可用容量。block 设备使用整盘节点；NAND/NOR 等 MTD 设备必须暴露代表整颗 flash 的 master mtd 节点，例如 `/dev/mtd0`，而不能只暴露分区 mtd 节点。若内核使用 mtdparts 拆分后看不到整颗 flash 的 master 节点，需要检查是否启用了 `CONFIG_MTD_PARTITIONED_MASTER=y`。没有 whole-chip mtd 节点时，GPT、IDBlock 或 full-flash 这类整盘写入不能可靠定位，会中止升级。

`bootloader` 对应 IDBlock 升级，只在编译 `RK_IDBLOCK_UPDATE` 时可用。升级时必须显式指定：

```
updateEngine --image_url=/userdata/update.img --update --upgrade=all,bootloader
```

IDBlock 写入时会根据介质类型和 Rockchip bootloader 格式生成启动数据，并按介质几何写入固定启动区域。为提高启动可靠性，IDBlock 会写入多份冗余拷贝；不同介质的起始位置、对齐方式和拷贝数量由对应存储后端决定，例如 SPI NOR 的 IDBlock 位于固定偏移区域，NAND 会结合 eraseblock 大小和坏块处理规则写入。

MTD 设备升级 IDBlock 同样要求存在整颗 NAND/NOR 的 mtd 节点，例如 `/dev/mtd0`。如果只存在 `/proc/mtd` 中的普通分区节点，updateEngine 无法把 IDBlock 按绝对地址写入启动区域，也不会退回到普通分区写入。

如果编译时关闭 GPT 或 IDBlock 升级功能，对应 item 会被跳过，不会退回到普通分区写入。不同存储介质的底层写入方式和限制见附录“updateEngine 存储写入实现说明”。

8.1.9 写入前检查

真正写入 flash 前，updateEngine 会执行检查。任一检查失败都会中止升级，避免写入一半后失败。

检查项	说明
挂载检查	普通目标分区不能处于挂载状态。
容量检查	写入范围不能超过目标设备容量。
GPT 布局检查	升级 parameter/GPT 时，如果某个分区起始位置变化，该分区必须也在本次升级列表中。
GPT 挂载分区检查	如果存在已挂载分区，新 GPT 中该分区起始位置不能变化，大小不能缩小。

例如升级包放在 `/userdata/update.img` 时，不要同时升级 userdata，否则可能出现如下错误：

```
precheck: userdata target /dev/block/by-name/userdata is mounted ... abort
```

8.2 自定义分区升级

当前版本的普通分区升级不需要在代码中维护固定分区表。只要升级包中存在对应 item，并且设备端能找到同名分区，就可以通过 `--upgrade=<name>` 指定升级。

例如要升级自定义分区 factory：

1. 确认设备分区表中存在 factory 分区。

block 设备通常需要存在：

```
/dev/block/by-name/factory
```

MTD 设备通常需要在 `/proc/mtd` 中存在同名分区，例如：

```
mtd5: 00400000 00020000 "factory"
```

2. 打包时把 factory 加入 package-file：

```
# NAME      Relative path
package-file package-file
factory      Image/factory.img
```

3. 在设备端执行升级：

```
updateEngine --image_url=/userdata/update.img \
    --misc=update \
    --upgrade=factory \
    --reboot
```

普通自定义分区默认走普通镜像写入流程。只有 `parameter` 和 `bootloader` 这类特殊 item 会进入 GPT/IDBlock 专用写入流程。不同存储介质的具体写入方式见附录“updateEngine 存储写入实现说明”。

9. 附录

9.1 固件打包工具

升级包制作方法见 升级包制作 章节。常用打包工具路径如下：

平台	工具路径	说明
Windows	`tools\windows\AndroidTool\rockdev`	修改 `package-file` 后执行 `mkupdate.bat`。
Linux	`tools/linux/Linux_Pack_Firmware/rockdev`	修改 `package-file` 后执行 `mkupdate.sh`。
差分包	`tools/linux/Linux_Diff_Firmware/`	执行 `mk-diff-ota.sh` 生成差分 OTA 包。

9.2 updateEngine 存储写入实现说明

updateEngine 根据目标分区所在的存储介质选择不同写入方式。普通 OTA 升级通常不需要关心底层细节，但在升级 NAND/NOR、UBI、parameter/GPT、bootloader/IDBlock、misc 或 A/B 信息时，需要理解这些限制。

介质	写入方式	说明
eMMC/UFS/SATA/NVMe 等 block 设备	通过 block 设备节点按 offset 写入	普通分区使用 by-name 分区节点；parameter/GPT 这类整盘数据使用整盘节点。
NAND	使用 `flash_erase`、`nandwrite`、`nanddump` 等 mtd-utils 工具	依赖工具处理坏块、OOB、ECC。
NOR	使用 MTD ioctl/read/write 等系统调用	NOR 没有 NAND 坏块跳过逻辑，但擦写仍以 eraseblock 为单位。
UBI	使用 UBI 卷接口写入	UBI 卷升级不直接写 raw MTD 分区。

NAND/NOR 写入 `misc`、A/B metadata、BCB 等小数据时，写入范围通常小于一个 eraseblock。为了保留同一个 eraseblock 内未修改的数据，updateEngine 会对这类非擦除块对齐的小数据写入执行 read-modify-write。

`parameter`/GPT、`bootloader`/IDBlock、full-flash 这类升级需要按绝对位置写入整盘或整颗 flash。block 设备需要整盘节点；NAND/NOR 等 MTD 设备必须暴露代表整颗 flash 的 master mtd 节点，例如 `/dev/mtd0`。如果系统只暴露分区 mtd 节点，updateEngine 无法可靠定位 GPT、IDBlock 或 full-flash 的绝对写入位置，会中止升级，不会退回到普通分区写入。

9.3 Misc 分区说明

Misc 分区是一个没有文件系统的分区，用于存放一些引导配置参数，现有结构如下，详见 external/recovery/core/bootmsg.c、external/recovery/core/slot.c 及相关头文件。

偏移地址	作用
2k	Linux A/B 分区引导信息
6k	格式化命令
16k	Recovery 系统与Normal 系统通信